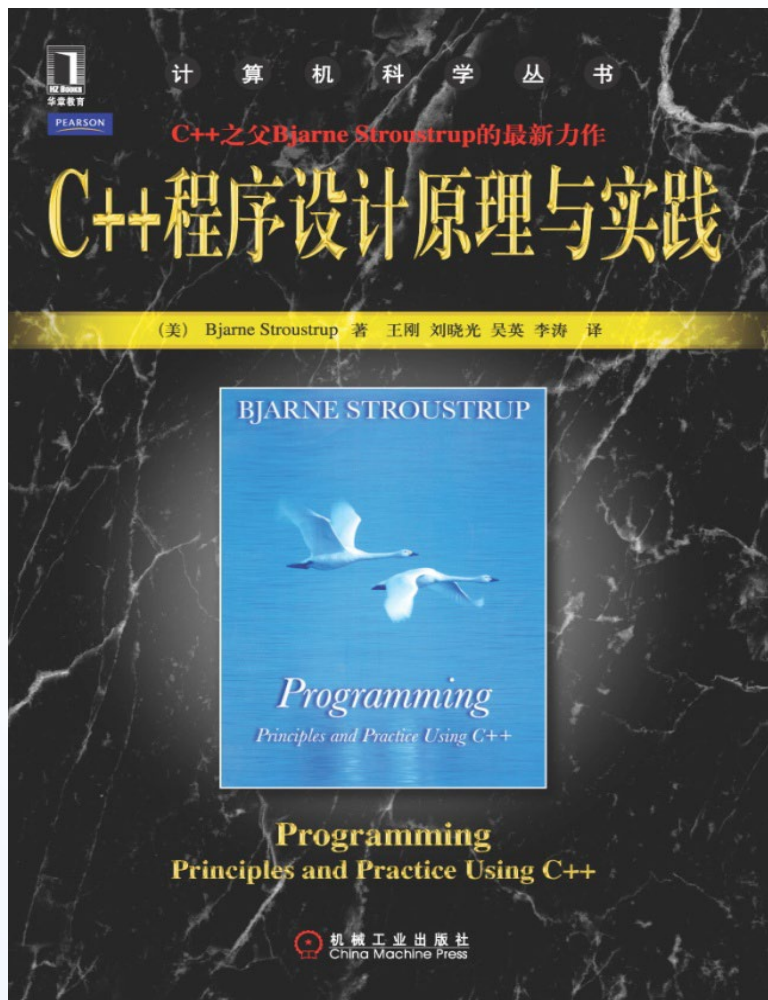


软件开发技术

第2章 软件开发的程序设计基础

浙江大学 机械工程学院 航空制造工程研究所

王青 李江雄



- 1、函数相关的技术细节
- 2、类相关的技术细节

基于C++语言讲解



01

函数相关的 技术细节

1.1声明和定义

◆ 声明

◆ 定义

◆ 为什么C++既提供声明，又提供定义？

◆ 变量和常量的声明/默认初始化

1.2头文件

◆ 实现声明和定义分离的一种措施，管理声明的一种手段

1.3作用域(scope)

◆ 作用域的作用

◆ 作用域的类型

◆ 分析变量和函数的作用域

◆ C++语法结构定义的一些常见作用域

◆ 能区别作用域的代码书写方法

1.4函数调用和返回

◆ 声明参数和返回类型

◆ 向函数传递参数的方式及对比

◆ 关于引用

1.5 命名空间

◆ 命名空间的作用

◆ 命名空间相关语法

课堂练习

课后作业

1.1 声明和定义

```
在编译下列程序时
int main()
{
    cout<<f(i)<<"\n";
}
```

会出现至少3个错误:

- (1) 不认识cout
- (2) 不认识f
- (3) 不认识i

```
在编译下列程序时
#include "std_lib_facillities.h"
int main()
{
    cout<<f(i)<<"\n";
}
```

会减少一个“不认识cout”错误

```
在编译下列程序时
#include "std_lib_facillities.h"
int f(int );
int main()
{
    int i=7;
    cout<<f(i)<<"\n";
}
```

编译通过，但连接时，会出错：
f (int) 没定义

第一次，我们没有告诉编译器，cout、f、i到底是什么

第二次，我们包含了“std_lib_facillities.h”，该文件内告诉了编译器，cout是什么，它具有<<操作

第三次，我们告诉了编译器，f是一个函数，其返回值为int，有一个调用参数，参数类型为int

声明 (declaration) 语句将名字引入到作用域，其作用是：

- 为命名实体（如变量、函数）指定一个类型
- （可选）进行初始化，例如：为变量指定初始值，或为函数指定函数体

如果一个声明还给出了声明的实体的完整描述，我们称之为**定义 (definition)**。

- 对于函数来说，给出了函数体的声明，则称之为定义
- 对于变量来说，为变量申请了内存空间，则称之为定义

定义

```
int a=7;  
vector<double> v;  
double sqrt(double d){/*...*/}  
int m;
```

仅仅是声明

```
extern int a;  
double sqrt(double d);
```

◆声明

告诉了编译系统如何使用该命名体，它只是提供了一个**接口**。至于接口内部的具体情况，编译系统并不知道。

对于变量名，声明告诉了编译系统，该命名体为一个什么类型的变量，编译系统只需按照该类型的变量规则进行检查就可。至于该变量存放在何处，编译系统并不需要知道。

对于函数，声明告诉了编译系统，该函数的返回值类型是什么，调用参数有哪些，各是什么类型。至于函数是如何实现的，编译系统不需知道。

◆定义

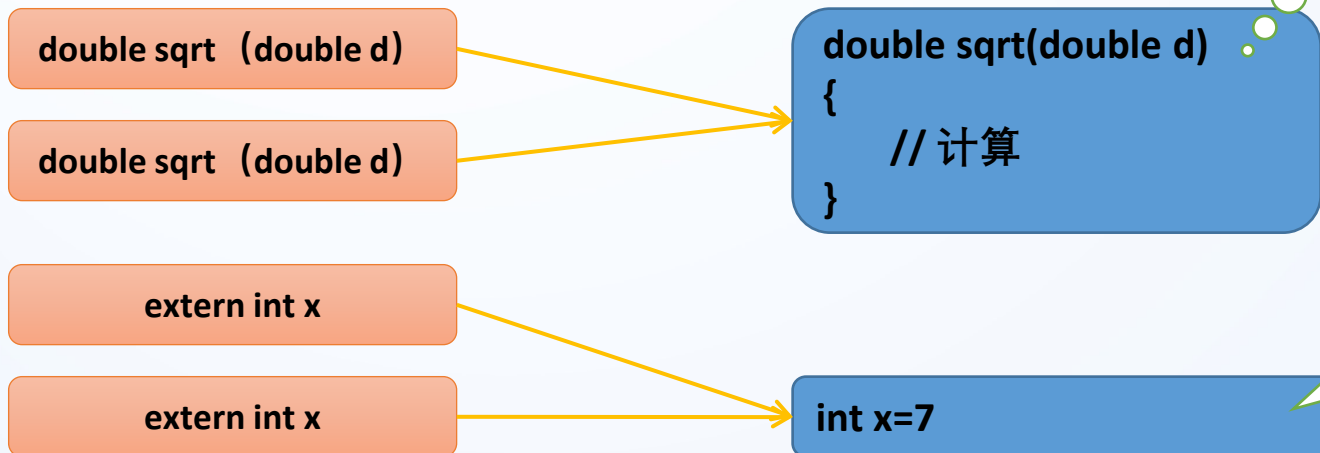
除告诉编译系统所有与声明相同的信息外，还告诉编译系统变量的内容（内存空间）放在何处，函数是如何实现的。在连接时，系统可以根据其内部机制到相应的内存空间找到变量指代的内容。

所以，**声明可以重复，而定义则只能一次。**

重复声明必须一致。

- 声明和定义例句：

- `int x=7;` `//definition`
- `extern int x;`
- `extern int x;`
- `extern double x;` `//error`
- `double sqrt(double);` `//declaration`
- `double sqrt(double) /*...*/` `//definition`
- `double sqrt(double);` `//2nd declaration`
- `int sqrt(double);` `//error 两次声明不一致`
- `int sqrt(double) /*...*/;` `//重复定义error`
- `double sqrt(double) /*...*/;` `//重复定义error`



多次声明，指向相同的函数体

多次声明，指向相同的内存空间

◆为什么C++既提供声明，又提供定义？

- 声明和定义的分离，反映出“如何使用一个实体”和“这个实体如何完成它应该做的事情”之间的根本区别，是信息隐藏的具体体现。
- 对于**变量**来说，声明仅仅提供了类型，而定义才能提供对象（内存空间）
- 对于**函数**来说，声明也只是给出了类型（参数类型和返回值类型），而定义才提供了函数体
- 定义消耗了内存空间，声明没有**

好处在于：

- 声明和定义的分离，可以让我们能将一个程序分成很多部分，分开编译。达到简化程序结构，提高程序可读性
- 能够处理好循环调用问题
- 定义只有一次，便于维护

01 函数相关的技术细节

//filename aa.cpp

```
double func_a(double a);  
double func_b(double b);  
double func_c(double c);
```

```
int main()
```

```
{  
/*
```

```
.....  
*/
```

```
double a = 6.0;  
double b = func_a(a);  
double c = func_b(b);  
double d = func_c(c);  
/*
```

```
.....  
*/  
}
```

声明

//filename bb.cpp

```
double func_a(double a);  
double func_b(double b);  
double func_c(double c);
```

```
int func_bb()
```

```
{  
/*
```

```
.....  
*/
```

```
double a = 6.0;  
double b = func_a(a);  
double c = func_b(b);  
double d = func_c(c);  
/*
```

```
.....  
*/  
}
```

声明

//filename cc.cpp

```
double func_a(double a) { /* ..... */ }  
double func_b(double b) { /* ..... */ }  
double func_c(double c) { /* ..... */ }
```

定义

//filename ee.cpp

```
double func_a(double a) { /* ..... */ }  
double func_b(double b) { /* ..... */ }  
double func_c(double c) { /* ..... */ }
```

```
int func_tt()
```

```
{  
/*
```

```
.....  
*/
```

```
double a = 6.0;  
double b = func_a(a);  
double c = func_b(b);  
double d = func_c(c);  
/*
```

```
.....  
*/  
}
```

定义

简化程序，不重复实现

```
double primary()  
{  
    //...  
    expression();  
    //...  
}  
  
double term()  
{  
    //.....  
    primary();  
    //...  
}  
  
double expression()  
{  
    //.....  
    term();  
    //.....  
}
```

定义于后面，在该作用域内，编译系统不认识。
循环调用，总有一个函数的定义在调用的后面。

```
double expression();  
  
double primary()  
{  
    //...  
    expression();  
    //...  
}  
  
double term()  
{  
    //.....  
    primary();  
    //...  
}  
  
double expression()  
{  
    //.....  
    term();  
    //.....  
}
```

前置声明，
处理循环
调用问题

◆变量和常量的声明

- 变量或常量声明需指定名字和类型，并可进行初始化
- 常量声明的语法与变量声明一样，差别在于类型之前多一个关键字const
- 常量必须初始化

变量声明

```
int a;                // 未初始化
double d=7;           // 使用赋值符=进行初始化
vector<int> vi(10);    //使用（）进行初始化
```

常量声明

```
const int x = 7;
const int x2(9);
const int y;          //错误，常量必须初始化
```

- 要养成初始化变量的习惯
- 记住“变量必须先初始化后使用”的原则，编译系统一般不会帮你检查。

默认初始化：

一般地，我们在定义自定义类型时，应该定义其初始化机制，以解决逐个初始化类中属性值的麻烦。但，C++不允许对内置类型设置默认初始化功能。

向量、字符串等在标准库中设置了默认初始化。

因此：

`Vector<int> vi(10);` 实际上对每个 `vi[i]`均进行了初始化。

`string s;` 实际上对s也进行了初始化。

1.2 头文件

声明与定义分离, 声明与定义还要一致, 声明可以重复, 重复的声明必须一致。

当一个命名体被多个程序使用时, 利用声明与定义分离的原则, 尽管其定义只有一个, 但在不同的程序中都有其声明。如何管理这些声明, 以保持声明的一致性?

在C++中, 对于别处定义的命名体的声明, 管理它们的关键是“头”(header)。一个头是声明的集合, 一般写在一个文件里, 该文件成为头文件。一般文件的后缀为.h。应用时, 只需要将该头文件用#include包含过来即可。

在C++编程时, 我们习惯将源文件放在*.cpp中, 而将其声明放在*.h文件中, 以方便其他程序使用。

包含头文件的效果, 相当于将被包含文件中的声明全部复制在文件的包含处。

```
#include "std_lib_facillities.h"
int f(int );
int main()
{
    int i = 7;
    cout<<f(i)<<"\n";
}
```

cout 声明已经写在 std_lib_facillities.h 中, 这里将该文件包含进来, 实际上就实现了对cout的声明。当然, 文件中还要很多其他标准实体的声明, 可供该程序使用。

1.3 作用域(scope)

命名实体的有效范围，我们称为其作用域。一般地，作用域是一个程序的文本区域。命名实体的声明只在其作用域内有效。出了其作用域，编译系统就不认识它了。



◆作用域的作用

保持名字的局部性，使之不影响声明于其他地方的名字。

```
void f(int x)
```

```
{
```

```
    int z = x+7;
```

```
}
```

```
int g(int x)
```

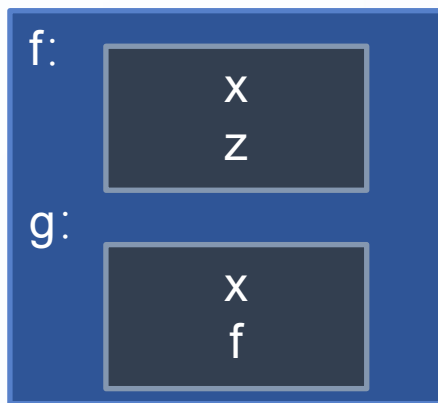
```
{
```

```
    int f=x+2;
```

```
    return 2*f;
```

```
}
```

全局作用域



全局作用域内有f, g

g的局部f的局部作用域内有x, z

作用域内有x, f

f中的x与g中的x不是同一对象；

g中的f与全局中的f也不是同一对象；

尽管它们同名，但并不冲突。

◆作用域 的类型

C++支持多种类型的作用域，帮助我们控制命名实体在何处可用。

- **全局作用域**：在任何其他作用域之外的程序区域
- **命名空间作用域**：限定于一个命名空间所规定的程序区域
- **类作用域**：一个类内的程序区域
- **局部作用域**：位于{ }之内或函数参数列表中的程序区域
- **语句作用域**：限定于一条语句中的程序区域，如for语句内的程序区域。

◆作用域分析例

```
// no r, i, or v here
class My_vector {
    vector<int> v;           // v is in class scope
public:
    int largest()
    {
        int r = 0;          // r is local (smallest nonnegative int)
        for (int i = 0; i<v.size(); ++i)
            r = max(r,abs(v[i])); // i is in the for's statement scope
        // no i here
        return r;
    }
    // no r here
};
// no v here
```

```
int x;      // global variable — avoid those where you can
int y;

int f()
{
    int x;    // local variable
    x = 7;    // the local x
    {
        int x = y;    // local x initialized by global y
        ++x;          // the x from the previous line
    }
    ++x;    // the x from the first line of f()
    return x;
}
```

◆C++语法结构定义的一些常见作用域

- 类中的函数：成员函数

```
class C {  
public:  
    void f();  
    void g()    // a member function can be defined within its class  
    {  
        // ...  
    }  
    // ...  
};  
  
void C::f()    // a member definition can be outside its class  
{  
    // ...  
}
```

这是一种最常见和最有用的情况。

- 类中的类：成员类(也称做嵌入式)

```
class C {  
public:  
    struct M {  
        // ...  
    };  
    // ...  
};
```

这只在复杂类中才有用，记住理想情况是保持类简短、简单。

- 函数中的类：局部类

```
void f()
{
    class L {
        // ...
    };
    // ...
}
```

应避免这种代码，如果你觉得需要一个局部类，那么你的函数可能太长了。

- 函数或块中的块：嵌套块

```
void f(int x, int y)
{
    if (x > y) {
        // ...
    }
    else {
        // ...
        {
            // ...
        }
        // ...
    }
}
```

c++中还提供了一种语言特性：命名空间，专门用于表达作用域

嵌套块是避免不了的，但要对复杂的嵌套保持警惕：它很容易隐藏错误。

◆能区别作用域的代码书写方法：

缩进

```
// dangerously ugly code
struct X {
void f(int x) {
struct Y {
int f() { return 1; } int m; };
int m;
m=x; Y m2;
return f(m2.f()); }
int m; void g(int m) {
if (m) f(m+2); else {
g(m+2); }}
X() {} void m3() {
}

void main() {
X a; a.f(2);
};
```



```
struct X{
    void f(int x){
        struct Y{
            int f(){return 1;}
            int m;
        };
        int m;
        m=x;
        Y m2;
        return f(m2.f());
    }
    int m;
    void g(int m){
        if(m)
            f(m+2);
        else{
            g(m+2);
        }
    }
}

X{}
void m3(){

}

void main(){
    X a;
    a.f(2);
}
```

1.4 函数调用和返回

◆ 声明参数和返回类型

函数声明:

函数名
double fat(int a, double d);
返回值类型 形参列表

函数定义:

double fat(int a, double d) { return a*d; }
 函数体

```
// 形参（参数）列表可以为空
double fct_2();
```

```
//无返回值的写法：
用void作返回值类型
```

```
//声明中，参数可以不命名
void fct_3(int,int);
```

```
//定义中，参数也可以不命名
//条件是函数体中不用该参数
void fct_4(int a,int)
{
/*
.....
*/
}
```

◆向函数传递参数

调用参数的过程就是用函数调用中指定的实际参数初始化函数的形式参数的过程

向函数传递参数有两种方法：

● 传值（称传值参数）

- 将参数的值拷贝一份，交给函数
- 函数体对传入参数无法修改
(分析参数作用域可以得出此结论)

```
int fct (int x, int y)
{
    ++x;
    --y;
    return 2*x;
}
int main()
{
    int a = 100;
    int b = 1000;
    int c = fct(a,b);
}
```

执行完后，
a=? , b=?

● 传引用（称引用参数）

- 将参数的值所占用的内存空间头地址交给函数。
- 函数可以修改参数的值。

```
int fct (int& x, int& y);
{
    ++x;
    --y;
    return 2*x;
}
int main()
{
    int a = 100;
    int b = 1000;
    int c = fct(a,b);
}
```

执行完后，
a=? , b=?

■传值方式和传引用方式对比：

● 传值方式

- 缺点：需要拷贝对象，因而消耗多余内存，降低程序执行效率；
- 优点：函数不改变调用的参数（实参）的值，能保持变量的局部性。

● 传引用方式

- 缺点：函数可以改变调用的参数（实参）的值，不能保证变量的局部性；
- 优点：不需拷贝对象，不消耗多余内存。

● 总结如下：

- 传值方式适合占内存小的参数，传引用方式适合占内存大的参数；
- 当需要在函数中改变实参的值时，可考虑使用传引用方式，但如果可用返回值来表达该参数的改变，应尽量用返回值表达，而不用引用；
- 如果传递的是占内存大的参数，而又确认函数不会对该大参数进行修改时，应采用**传常量引用方式**。

01函数相关的技术细节

三种方式可以混合使用：

```
void g(int a, int& r, const int& cr)
{
    ++a;      // change the local a
    ++r;      // change the object referred to by r
    int x = cr; // read the object referred to by cr
}
```

传值、传常量引用可以直接用数值初始化，
而传引用不能

```
int main()
{
    int x = 0;
    int y = 0;
    int z = 0;

    g(x,y,z);    // x==0; y==1; z==0
    g(1,2,3);    // error: reference argument r needs a variable to refer to
    g(1,y,3);    // OK: since cr is const we can pass a literal
}
```

大对象参数，尽量用传常量引用方式

```
void print(const vector<double>& v)    // pass-by-const-reference
{
    cout << "{ ";
    for (int i = 0; i < v.size(); ++i) {
        cout << v[i];
        if (i != v.size() - 1) cout << ", ";
    }
    cout << " }\n";
}
```

小对象，需要函数修改其值，也只能用传引用方式

```
void swap(double& d1, double& d2)
{
    double temp = d1;    // copy d1's value to temp
    d1 = d2;              // copy d2's value to d1
    d2 = temp;            // copy d1's old value to d2
}
```

大对象，用传引用方式

```
void init(vector<double>& v)    // pass-by-reference
{
    for (int i = 0; i < v.size(); ++i) v[i] = i;
}
```

大对象，用传引用方式

```
void larger(vector<int>& v1, vector<int>& v2)
    // make each element in v1 the larger of the corresponding
    // elements in v1 and v2;
    // similarly, make each element of v2 the smaller
{
    if (v1.size() != v2.size()) error("larger(): different sizes");
    for (int i = 0; i < v1.size(); ++i)
        if (v1[i] < v2[i])
            swap(v1[i], v2[i]);
}
```

◆关于引用

让我们从更技术化的角度来探讨一下引用。引用是这样一种语法机制，它允许用户为一个对象声明一个新的名字。例如，`int&` 是一个整型对象的引用，因此，我们可写出如下代码：

```
int i = 7;

int& r = i;    // r is a reference to i
r = 9;         // i becomes 9
i = 10;
cout << r << ' ' << i << '\n';    // write: 10 10
```



也就是说，任何对 `r` 的使用实际上使用的是 `i`。

引用的一个用途是作为简写形式。例如，我们可能用到如下二维向量：

```
vector< vector<double>> v;    // vector of vector of double
```

如果我们只是需要这个元素的值，那么可以声明下面这个变量：`double val = v[f(x)][g(y)];`

然后多次使用 `val` 即可。

如果我们既要从 `v[f(x)][g(x)]` 中读取值，又要向它写入值引用就派上用场了：

```
double& var = v[f(x)][g(y)];
```

通过 `var`，我们既可以从 `v[f(x)][g(x)]` 中读取值，也可以向它写入值。

• 1.5 命名空间

命名空间用于对声明进行分组管理

```
namespace TextLib {  
    class Text { /* ... */};  
    class Glyph { /* ... */};  
    class Line { /* ... */};  
    // ...  
}
```

```
namespace Graph_lib {  
    struct Color { /* ... */};  
    struct Shape { /* ... */};  
    struct Line : Shape { /* ... */};  
    struct Function : Shape { /* ... */};  
    struct Text : Shape { /* ... */};  
    // ...  
    int gui_main() { /* ... */}  
}
```

TextLib命名空间和Graph_lib命名空间里均定义了类：Text。

由于两个Text类分别属于不同的命名空间，两者作用域不重叠，因而两者不冲突。
使用时为了区别，可以带上命名空间，如图TextLib::Text和Graph_lib::Text：

◆命名空间相关语法

• (1) 定义

```
标识符关键字    名称
namespace TextLib {
    class Text { /* ... */ };
    class Glyph { /* ... */ };
    class Line { /* ... */ };
    // ...
}
```

• (2) 使用

- 命名空间：： 空间内的命名体；如
 - TextLib::Text,
 - TextLib::Line （全限定名方式）

- 使用using 指令

- using namespace TextLib,
- 其含义为如果在本作用域中没有发现某个名字的声明，那么请到命名空间TextLib中去找。

- using声明：using TextLib::Text

当后面出现 Text时，指的是TextLib中的Text。

● 谨慎使用using

● 尽量不要在头文件中使用

using指令

三个函数 `swap_v(int, int)`

`swap_r(int&, int&)`

`swap_cr(const int&, const int&)`

每个函数有如下函数体：

```
{ int temp; temp = a, a=b; b=temp; }
```

其中 `a` 和 `b` 是参数名。

尝试用如下代码调用这三个函数：

```
int x = 7;
int y = 9;
swap_?(x,y); // replace ? by v, r, or cr
swap_?(7,9);
const int cx = 7;
const int cy = 9;
swap_?(cx,cy);
swap_?(7.7,9.9);
double dx = 7.7;
double dy = 9.9;
swap_?(dx,dy);
swap_?(dx,dy);
```

哪个调用能编译通过？为什么？

对于编译通过的`swap`函数，在其调用后，哪个参数真正被交换过了？

编写一个程序，由一个文件组成，其中包含三个名字空间 X、Y 和 Z，使得如下 main() 函数能正确运行：

```
int main()
{
    X::var = 7;
    X::print();    // print X's var
    using namespace Y;
    var = 9;
    print();       // print Y's var
    {
        using Z::var;
        using Z::print;
        var = 11;
        print();   // print Z's var
    }
    print();       // print Y's var
    X::print();    // print X's var
}
```

每个名字空间需定义一个名为 var 的变量和一个名为 print() 的函数，该函数用 cout 输出恰当的 var 值。



02

类相关的 技术细节

2.1 类

◆ 访问类成员的方法

◆ 接口和实现

◆ 结构

◆ 构造函数

◆ 析构函数

2.2 枚举类型

◆ 枚举类型的使用

◆ 枚举类型的用途

◆ 枚举量的作用域

2.3 运算符重载

2.4 类接口

2.5 一个类的代码（Date类）

2.6 基类和派生类

课后作业

2.1 类

一个类就是一个用户自定义类型，由一些内置类型和其他自定义类型的对象以及一些函数组成。一般地，我们称这些用来定义类的组成部分为类的成员（member），一个类可以有0个成员，也可以有多个成员。

类的成员大致可以分为两类：

- 一类用来表示类的对象的属性，称类的属性，一般是变量的形式
- 另一类用来表示类的对象的操作，称为类的方法，一般是函数

```
class X{  
public:  
    int m;  
    int mf(int v){  
        int old=m;  
        m=v;  
        return old;  
    }  
};
```

```
class Y{  
private:  
    int m;  
public:  
    X x;  
    int xm(){return x.m;}  
    int ym(){return m;}  
};
```

■ 从软件开发（建模）的角度，类是对一类具有共同特性的对象的抽象，它表达了这一类对象的共同特性和行为。

■ 从编程的角度，类是一种自定义类型，它指出这种类型的对象如何表示、如何创建、如何使用、如何销毁。

◆访问类成员的方法

- 对象名.成员名
- `X xvar;`
- `int a = xvar.m;` //读成员变量
- `xvar.m = 9;` //写成员变量
- `int m = xvar.m+1;` //读成员变量
- `int mf = xvar.mf(8);` //调用成员函数
- `Y yvar;`
- `int xm = yvar.xm();` //调用成员函数
- `int ym = yvar.ym();` //调用成员函数

- `int m = yvar.xm()+yvar.ym();`
- `int g = yvar.x.m;` //正确
- `yvar.x.m=9;` //正确
- `int f = yvar.m;` //错误, 为什么?
- `yvar.m=9;` //错误

```
class X{  
public:  
    int m;  
    int mf(int v){  
        int old=m;  
        m=v;  
        return old;  
    }  
};
```

```
class Y{  
private:  
    int m;  
public:  
    X x;  
    int xm(){return x.m;}  
    int ym(){return m;}  
};
```

◆ 接口和实现

一般可以将类分为接口和实现两个部分

- 接口部分：曝露给类的用户的部分，用户可以直接访问它（读、写、调用）
 - 实现部分：对用户进行信息隐藏的部分，用户只能通过接口间接访问它。
 - 接口部分声明的成员称为公有成员用public表示
 - 实现部分声明的成员称为私有成员用private表示
- 可以将类理解为下列形式：

```
class X { // this class's name is X
public:
    // public members:
    // - the interface to users (accessible by all)
    // functions
    // types
    // data (often best kept private)
private:
    // private members:
    // - the implementation details (used by members of this class only)
    // functions
    // types
    // data
};
```

另外还有一部分特殊接口称为保护部分protect，只对其朋友曝露（友元）

```
class Y{
    int a;
    double d(){};
private:
    int m;
public:
    X x;
    int xm(){return x.m;}
    int ym(){return m;}
};
```

类成员缺省为私有的

为什么要将接口和实现分离

类Date1中，y、m、d均为公有成员用户可以访问。如：

```
Date1 today;
```

```
today.y = 2011;
```

```
today.m = 15;
```

```
today.d = 45;
```

显然，该日期是不符合逻辑的，谁负责检查呢？只能靠用户自己。如果说，将y、m、d设计为private成员，不运行用户直接访问，那么，我们可以让对象自己进行检查，以保证其合法性。如Date2；

```
class Date1{
public:
    int y,m,d;
    Date(){y=2000;m=1;d=1;}
    Date(int y,int m,int d);
    void add_day(int n);
    int check();
}
```

```
class Date2
{
    int y,m,d;
public:
    int year(){return y;}
    int month(){return m;}
    int day(){return r;}
    void setyear(int y){/*...*/}
```

```
void setmonth(int m){/* 检查...*/}
void setday(int r){/*...检查*/}
Date(){y=2000;m=1;d=1;}
Date(int y,int m,int d);
void add_day(int n);
int check();
}
```

◆ 结构

结构是一种特殊的类，其成员全部为公有成员，不能定义私有成员。因而内部不需要再写出 public 标识符。

```
struct X {  
    int m;  
    // ...  
};
```

等价于

```
class X {  
public:  
    int m;  
    // ...  
};
```

◆ 构造函数

是与类同名的成员函数，专门用于对类的对象进行初始化。一般在给对象申请内存空间时（即创建类的对象时）调用。

```
class Date {  
public:  
    int y,m,d;  
    //无参数构造函数，缺省初始化Date  
    Date() {y=2000;m=1;d=1;}  
    //含参数构造函数，用参数初始化Date  
    Date(int y,int m,int d);  
    void add_day(int n); //increase the Date by n days;  
}
```

使用时：

//缺省调用 Date () 构造函数

Date my_birthday;

//调用Date (int y,int m,int d)构造函数

Date today (2011, 12, 5) ;

■使用构造函数的好处:

➤不会忘记对新创建的对象进行初始化

- 如果定义了无参数的构造函数, 创建对象时, 即使用户没有主动调用构造函数, 系统自动调用它, 实现初始化
- 如果没有定义无参数的构造函数, 创建对象时, 如果用户没有主动调用构造函数, 编译器会捕捉到该错误, 要求你必须调用一个合适的构造函数

➤简化初始化代码, 使程序结构性更好, 便于维护

用户不必在创建每个对象时, 挨个为其成员进行初始化, 只需调用构造函数即可。既简化了代码, 又保证初始化代码的一致性

➤可以在构造函数中设置检查功能, 检查初始化数据的合法性

例如: 对于语句: `Date today (12,24,2000)` ;

如果, 我们在Date的构造函数 `Date (int y,int m,int r)` 中设置了日期合法性检查代码如下:

`r` 必须大于0。

1、3、5、7、8、10、12月`r`必须小于32;

4、6、9、11月`r`必须小于31;

闰年2月, `r`小于30, 否则`r`小于29

`m` 必须大于0, 小于13

就可以在创建对象时就判断出today的初始化值是不合法的。

◆析构函数

在类名前加“~”的成员函数，专门用于对象销毁时作后置处理，常用于释放对象成员所占用的内存。

```
class Calendar{  
public:  
    Date date;  
    Image* pImage;  
    Text* pText;  
  
    Calendar(){  
        pImage = NULL;  
        pText = NULL;  
    }  
  
    ~Calendar () {  
        if(pImage) delete pImage;  
        if(pText)delete pText;  
        cout<< “ I am out” << “\n” ;  
    }  
}
```

对象离开其作用域时，自动销毁，此时自动调用析构函数，我们可以在析构函数中将pImage指向的内存空间清除，将pText指向的内存空间清除。当然，还可以作其他后置处理工作。

2.2 枚举类型

枚举 (enumeration, enum) 是一种最为简单的自定义类型，它指定一个值的集合。这些值用符号常量表示。称为枚举量 (enumerator)。

```
enum Month {
    jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
};
```

含义：定义一个类型Month，其对象的取值只能是jan，…dec中的一个。

jan…dec可以理解为常量，其值依次为1,2,3…12。

当然，我们可以不为jan赋值，这样的话，jan…dec值依次为0,1,2,3…11。

我们还可以为每个常量赋值，如：

```
enum Month{
    jan=1,feb=2,mar=43,apr=4,may=5,jun=6,jul=7,aug=8,sep=9,oct=10,nov=11,dec=12
}
```

◆ 枚举类型的使用

```
Month m = feb;
Month m = feb;
Month m = 7; // 错误，整型不能隐式转换为枚举型
int m = mar; // 正确，枚举型可以隐式转换为整型
Month m = Month(7); // 正确，枚举型可以显式转换为整型
Month m = Month(9999); // 语法正确，但不符合逻辑，编译系统查不出
```

◆枚举类型的用途

枚举类型可用于任何需要一组相关的命名整型常量的地方。如：

- 一年内的月份：
- 一周的天：星期天，星期一，…星期六
- 方位：东南西北
- 标准颜色：红橙黄绿蓝
- 学校里的系别：机械系、电机系、…
- 人的性别：male, female

◆枚举量的作用域

枚举量如jan/feb/mar…的作用域不局限在枚举类型内，而是与其枚举类型有着相同的作用域（与类的成员变量不同）。如：

```
enum Traffic_sign{red, yellow, green}
```

```
int var = red;
```

也就是说，只有定义了枚举类型，就相当于定义了一系列符号常量，这些符号常量的作用域定义它们的枚举类型相同。哪里要用它，这些符号常量就跟到哪里。

2.3 运算符重载

```
class Vector{// 矢量
    double i;
    double j;
    double k;
}
// 现要实现矢量的四则运算
矢量加法
矢量减法
矢量乘法 (叉乘)
矢量除法 (矢量除以标量)
当然可以写成
```

```
class Vector{// 矢量
    double i;
    double j;
    double k;
public:
    Vector plus (Vector v);
    Vector minus(Vector v);
```

```
Vector crossmulti(Vector v);
Vector div(double d);
```

```
}
```

使用时:

```
Vector v(1,0,0);
Vector a(0,1,0);
Vector b(0,0,1);
v = a.plus(b);
v= a.minus(b);
v = a.crossmulti(b);
v= a.div(5.0);
```

还可以写成:

```
class Vector{// 矢量
    double i;
    double j;
    double k;
public:
```

```
Vector operator + (Vector v);
Vector operator - ( Vector v);
Vector operator * ( Vector v);
Vector operator / (double d);
```

```
}
```

使用时:

```
Vector v(1,0,0);
Vector a(0,1,0);
Vector b(0,0,1);
v = a+b;
v= a-b;
v = a*b;
v= a/5.0;
```

再看前面定义的枚举类型Month，若要实现前一月，后一月的运算，也可以用--，++操作符。

Month operator--(Month& m)

```
{  
    m = (m==Jan)?Dec:Month(m-1);  
    return m;  
}
```

Month operator++(Month& m)

```
{  
    m = (m==Dec)? Jan : Month(m+1);  
    return m;  
}
```

```
Month m = apr;  
--m;           //m become Mar  
--m;           //m become Feb  
--m;           //m become Jan  
--m;           //m become Dec
```

```
Month m = Sep;  
++m;           // m becomes Oct  
++m;           // m becomes Nov  
++m;           // m becomes Dec  
++m;           // m becomes Jan ("wrap around")
```

运算符重载(operator overloading)是指在枚举或类对象上定义C++的操作符。其作用是为用户自定义类型提供习惯的符号表示方法。

注意之处：

- 重载运算符应该保持其原有意义；以体现“习惯”两字
- 一个重载运算符至少作用于一个自定义类型

2.4 类接口

类的公共接口与实现部分应该是分离的，但如何来设计一个好的接口呢？好的公共接口和乱糟糟的接口之间有什么区别呢？我们通过实例来回答这个问题，首先我们先来看一下接口设计的一般原则：

- 保持接口的完整性
- 保持接口的最小化
- 提供构造函数
- 支持（或者禁止）拷贝
- 使用类型来提供完善的类型检查
- 支持不可修改的成员函数
- 在析构函数中释放所有资源

□ 参数类型：

在前面Date类定义中使用了3个int型作为参数，会存在什么问题？

```
Date d1(4, 5, 2011);
```

```
Date d2(2011, 4, 5);
```

这类问题通常的解决方案是**使用类型系统**。

```
class Date{
public:
    enum Month{
        jan = 1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};
    Date(int y, Month m, int d);
private:
    int y;           // year
    Month m;
    int d;           // day
};
```

□ 拷贝

编程需要创建对象，即需要考虑初始化和构造函数，同时还需要赋值，在类设计中即为拷贝构造函数和拷贝赋值操作符。

```
class Date{
public:
    enum Month{
        jan = 1, feb, mar, apr, may, jun,
        jul, aug, sep, oct, nov, dec};
    Date(int y, Month m, int d);
    Date(const Date& rhs){
        this->y = rhs.y;
        this->m = rhs.m;
        this->d = rhs.d;
    }
    Date& operator =(const Date& rhs){
        if(&rhs == this)
            return *this;
```

```
        y = rhs.y;
        m = rhs.m;
        d = rhs.d;
        return *this;
```

```
    }
private:
    int y;           // year
    Month m;
    int d;           // day
};
```

```
Date d(2011, Date::may, 4);
// 使用拷贝构造函数
Date today(d);
// 使用拷贝赋值操作符
today = d;
```

□ const 成员函数

将类成员分为公共接口和实现两个部分的一个重要原因是防止用户不按规定修改对象的成员变量。但是，类成员函数是否也会修改成员变量的值呢？用户不知道。为了明确告诉用户某个成员函数不会修改对象的属性，可以使用const 成员函数。

```
void some_function(Date& d,const Date& start_of_term)
```

```
{
```

```
    int a = d.day();
```

```
    int b = start_of_term.day();
```

```
    d.add_day(3);
```

```
    start_of_term.add_day(3);
```

```
}
```

some_function 函数要求函数不改变参数*start_of_term*的值。

但是，函数中*start_of_term*调用了其成员函数*day()*和*add_day()*。

*day()*和*add_day()*是否会改变其对象中某个属性的值呢？我们不知道，编译系统也不知道。为保险起见，编译器会假设该两函数会改变*start_of_term*对象的某些属性的值，因它认为与函数的声明不符，因而报错。

为了明确哪些函数不会改变对象的属性，C++提供const 成员函数的概念将类的操作分为：可更改对象属性和不可更改对象属性的函数两类。

```
class Date {  
public:  
    // ...  
    int day() const;           // const member: can't modify the object  
    Month month() const;      // const member: can't modify the object  
    int year() const;         // const member: can't modify the object  
  
    void add_day(int n);       // non-const member: can modify the object  
    void add_month(int n);     // non-const member: can modify the object  
    void add_year(int n);      // non-const member: can modify the object  
private:  
    int y;                    // year  
    Month m;                  // month  
    int d;                    // day of month  
};  
  
Date d(2000, Date::jan, 20);  
const Date cd(2001, Date::feb, 21);  
  
cout << d.day() << " — " << cd.day() << endl; // OK  
d.add_day(1);    // OK  
cd.add_day(1);   // error: cd is a const
```

不可更改操作

可更改操作

常量不能使用可
更改操作

□ 类成员和“辅助函数”

在保证类接口完整性的情况下试图最小化类接口时，不得不忽略大量有用的操作，但如果一个函数可以简单、优美、高效地实现为一个独立函数（即实现非成员函数）时，就应该将其放在类外面。

这样处理的优点：

- ◆ 函数中的错误不会直接破坏类对象中的数据；
- ◆ 如果类描述改变，只有直接访问描述的函数才需要重写。（这也是保持接口最小化的另一个重要原因）

```
bool operator==(const Date& a, const Date& b)
{
    return a.year()==b.year()
        && a.month() == b.month()
        && a.day() == b.day();
}
bool operator!=(const Date& a, const Date& b)
{
    return !(a==b);
}
```

这类函数也称辅助函数(helper function)或便利函数(convenience function)。

2.5 一个类的代码（Date类）

声明部分：放在头文件Chrono.h中

```
// file Chrono.h

namespace Chrono {

class Date {
public:
    enum Month {
        jan=1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec
    };

    class Invalid {}; // to throw as exception

    Date(int y, Month m, int d); // check for valid date and initialize
    Date(); // default constructor
    // the default copy operations are fine

    // nonmodifying operations:
    int day() const { return d; }
    Month month() const { return m; }
    int year() const { return y; }

    // modifying operations:
    void add_day(int n);
    void add_month(int n);
    void add_year(int n);
private:
    int y;
    Month m;
    int d;
};
```

```
bool is_date(int y, Date::Month m, int d); // true for valid date

bool leapyear(int y); // true if y is a leap year

bool operator==(const Date& a, const Date& b);
bool operator!=(const Date& a, const Date& b);

ostream& operator<<(ostream& os, const Date& d);

istream& operator>>(istream& is, Date& dd);

} // Chrono
```

Date类的辅助函数

定义在 Chrono. cpp 中:

```
// Chrono.cpp

namespace Chrono {

// member function definitions:

Date::Date(int yy, Month mm, int dd)
    : y(yy), m(mm), d(dd)
{
    if (!is_date(yy,mm,dd)) throw Invalid();
}

Date& default_date()
{
    static Date dd(2001,Date::jan,1); // start of 21st century
    return dd;
}

Date::Date()
    :y(default_date().year()),
    m(default_date().month()),
    d(default_date().day())
{
}

void Date:: add_day(int n)
{
    // ...
}

void Date::add_month(int n)
{
    // ...
}
```

```
void Date::add_month(int n)
{
    // ...
}

void Date::add_year(int n)
{
    if (m==feb && d==29 && !leapyear(y+n)) { // beware of leap years!
        m = mar; // use March 1 instead of February 29
        d = 1;
    }
    y+=n;
}

// helper functions:

bool is_date(int y, Date::Month m, int d)
{
    // assume that y is valid

    if (d<=0) return false; // d must be positive

    int days_in_month = 31; // most months have 31 days

    switch (m) {
    case Date::feb: // the length of February varies
        days_in_month = (leapyear(y)) ? 29 : 28;
        break;
    case Date::apr: case Date::jun: case Date::sep: case Date::nov:
        days_in_month = 30; // the rest have 30 days
        break;
    }

    if (days_in_month<d) return false;

    return true;
}
```

```
bool leapyear(int y)
{
    // see exercise 10
}

bool operator==(const Date& a, const Date& b)
{
    return a.year()==b.year()
        && a.month()==b.month()
        && a.day()==b.day();
}

bool operator!=(const Date& a, const Date& b)
{
    return !(a==b);
}

ostream& operator<<(ostream& os, const Date& d)
{
    return os << '(' << d.year()
        << ',' << d.month()
        << ',' << d.day() << ')';
}

istream& operator>>(istream& is, Date& dd)
{
    int y, m, d;
    char ch1, ch2, ch3, ch4;
    is >> ch1 >> y >> ch2 >> m >> ch3 >> d >> ch4;
    if (!is) return is;
    if (ch1!='(' || ch2!=',' || ch3!=',' || ch4!=')') { // oops: format error
        is.clear(ios_base::failbit); // set the fail bit
        return is;
    }

    return is;
}
```

```
enum Day {
    sunday, monday, tuesday, wednesday, thursday, friday, saturday
};

Day day_of_week(const Date& d)
{
    // ...
}

Date next_Sunday(const Date& d)
{
    // ...
}

Date next_weekday(const Date& d)
{
    // ...
}

} // Chrono
```

2.6 基类和派生类

依赖于3个关键的语言机制：

□ **派生 (derivation)** : 从一个类构造另一个类的方法，例如：Circle类从Shape类派生，则Circle是某种Shape，或者Shape是Circle的基类，派生类除了自身的成员外，还包括基类的成员，这通常称为“**继承**”，派生类继承了基类的所有成员，被称为子类，基类被称为父类。

```
class Shape
{
public:
    void draw();
    void move();
};

class Circle : public Shape
{ public: /* ..... */ };

Circle c;
c.draw();
c.move();
```

❑ **虚函数 (virtual function)** : 在基类中定义了一个函数, 在派生类中有一个类型和名称完全一样的函数, 当用户调用基类函数时, 实际上调用的是派生类中的函数。这种通常称为“**运行时多态**”, 或**动态分派**, 应为具体调用哪个函数根据运行时实际使用的对象类型确定。

```
class Shape{  
public:  
    virtual void draw();  
    void move();  
};  
class Circle : public Shape{  
public:  
    virtual void draw();  
};  
class Line : public Shape{  
public:  
    virtual void draw();  
};  
  
Shape * pCir = new Circle;  
pCir->draw();  
pCir->move();  
  
Shape* pLine = new Line;  
pLine->draw();  
pLine->move();
```

□ **私有和保护成员 (private and protected member)** : 保持类的实现细节为私有的, 以便保护他们不被直接访问, 简化维护操作, 这通常称为“**封装 (encapsulation)**”。

在C++中, 类成员可以是:

- ◆ **私有的 (private)** : 只能被所属类成员使用;
- ◆ **受保护的 (protected)** : 只能被所属类及派生类成员使用;
- ◆ **公有的 (public)** : 可以被所有函数使用。

基类也可以是私有的, 受保护的, 公有的:

- ◆ 如果类D的一个基类是private的, 基类的public和protected成员的名字只能被类D的成员使用;
- ◆ 如果类D的一个基类是protected的, 基类的public和protected成员的名字只能被类D及其派生类的成员使用;
- ◆ 如果类D的一个基类是public的, 基类的名字可以被所有函数使用。



THANK YOU

浙江大学机械工程学院航空制造工程研究所
王青 李江雄